

Computer Programming 143 – Lecture 13

Arrays I

Faculty of Engineering



Copyright

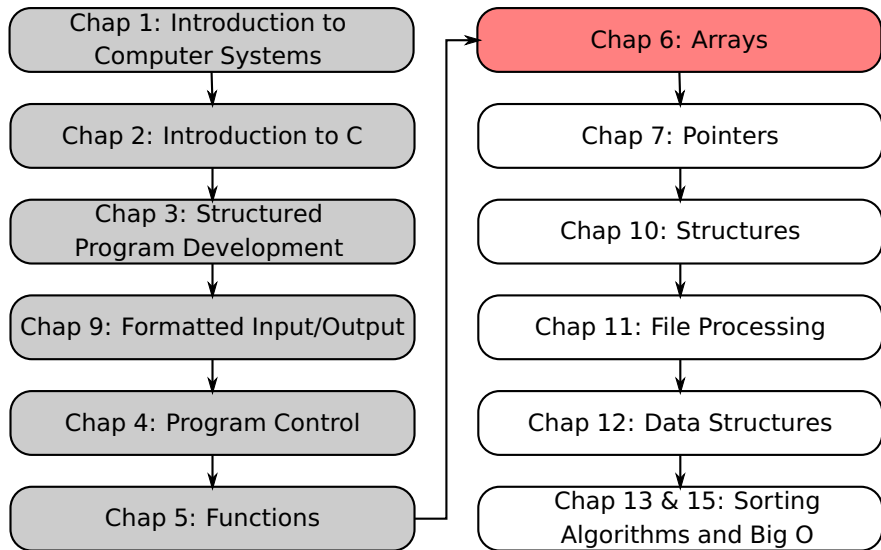
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

- 1 Introduction (6.1)
- 2 Arrays (6.2)
- 3 Declaring Arrays (6.3)
- 4 Examples Using Arrays (6.4)

6.1 Introduction

Definition of array

- Structure that stores related data items of the same data type
- Fixed size throughout program execution
 - Dynamic data structures discussed in Chapter 12

6.2 Arrays I

Array

- Group of consecutive memory locations
- Same name and type

To refer to a specific element

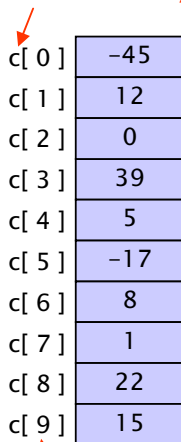
- Array name
- Position number (index)

Format:

name[position number];

- First element is at position 0
- **n** element array named **c**:
 - **c[0], c[1] ... c[n-1]**

Name of array



c[0]	-45
c[1]	12
c[2]	0
c[3]	39
c[4]	5
c[5]	-17
c[6]	8
c[7]	1
c[8]	22
c[9]	15

Position number

6.2 Arrays II

Array elements are like normal variables

- Assign a value of 3 to the first element of array **c**:

```
c[ 0 ] = 3;
```

- Display the sum of the first 3 elements of **c**:

```
printf( "%d", c[ 0 ] + c[ 1 ] + c[ 2 ] );
```

- Halve the 7th element of **c** and store the result in variable **x**:

```
x = c[ 6 ] / 2;
```

- Can perform operations in index. If **a** = 3 and **b** = 2, then

```
c[ a + b ] += 2;
```

would increment the 6th element of **c** by 2

6.2 Arrays III

Operator precedence

Operators	Associativity	Type
[] ()	left to right	highest
++ -- ! (type)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical and
	left to right	logical or
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

6.3 Declaring Arrays

When declaring arrays, specify

- Type of array
 - Name
 - Number of elements
- arrayType arrayName[numberOfElements];*

Examples:

```
int c[ 10 ];  
float myArray[ 3284 ];
```

Declaring multiple arrays of same type

- Format similar to regular variables

Example:

```
int b[ 100 ], x[ 27 ];
```

6.4 Examples Using Arrays

Initialising arrays

- Like other variables, array elements are not initialised automatically at declaration
- Initialise elements by assigning values one by one (Fig. 6.1 in D&D)
- Or use an initialiser list:

```
int n[5] = { 1, 2, 3, 4, 5 };
```

- If initialiser list shorter than array – remaining elements become 0

```
int n[5] = { 0 };
```

- All 5 elements of **n** are initialised to 0
- If size omitted, initialisers determine it

```
int n[ ] = { 1, 2, 3, 4, 5 };
```

- 5 initialisers, therefore 5-element array

6.4 Examples Using Arrays

Problem

- Initialize an array with the even integers from 2 to 10 and then display the array.

6.4 Examples Using Arrays

Pseudocode

for each integer j from 1 to 5 (j is loop counter)
*set array element = $2*j$*

for each integer j from 1 to 5
print array element j

6.4 Examples Using Arrays

Example

/ Fig. 6.3 in Deitel&Deitel : Initialising the elements of array s to the even integers from 2 to 10 */*

#include <stdio.h>

#define SIZE 5 *// maximum size of array*

int main(**void**) {

int s[SIZE]; *// array has SIZE elements*

int j; *// counter*

for (j = 0; j < SIZE; j++) { *// set the values*

 s[j] = 2 + 2 * j;

 } *// end for*

 printf("%s%13s\n", "Element", "Value");

for (j = 0; j < SIZE; j++) {

 printf("%7d%13d\n", j, s[j]);

 } *// end for*

return 0; *// indicates successful termination*

} *// end main*

6.4 Examples Using Arrays

Output

Element	Value
0	2
1	4
2	6
3	8
4	10

6.4 Examples Using Arrays

Symbolic constants

```
#define SIZE 10
```

- Is a C preprocessor directive
- Defines a symbolic constant **SIZE** of value 10
- Preprocessor replaces all occurrences of **SIZE** in the text with 10 before program is compiled
- Makes program scalable – value is changed in only one place in the text

6.4 Examples Using Arrays

Survey example: problem statement

Forty students were asked to rate the quality of the food in the student cafeteria on a scale of 1 to 5 (1 means awful and 5 means excellent). Place the 20 responses in an integer array and summarise the results of the poll.

6.4 Examples Using Arrays

Pseudocode

Declare and initialise an array called bins to keep count of each rating score. Each element of bins will represent a score, and will count the number of times the score occurs.

Declare and initialise an array called responses to capture all the responses

For each response in the responses array

Determine the rating score

Increment the corresponding element in the bins array

For each rating score (i.e. 1 to 5)

Print the corresponding element in the bins array.

6.4 Examples Using Arrays

```
/* Student poll program; Modified from Fig. 6.5 in Deitel & Deitel */
#include <stdio.h>
#define RESPONSE_SIZE 20 // number of responses
#define FREQUENCY_SIZE 5 // number of possible ratings

int main( void ) { // function main begins program execution
    int rating; // temporary, stores rating of current response
    /* frequency array to keep count of number of occurrences per rating
    * frequency[0] will keep track of the number of "1" responses,
    * frequency[n] will keep track of the number of "n+1" responses
    */
    int frequency[ FREQUENCY_SIZE ] = {0};
    //initialise frequency (set all to 0)

    // responses hard-coded in the responses array for this example
    int responses[ RESPONSE_SIZE ] = {1,2,5,4,3,5,2,1,3,1,4,3,3,3,2,
        3,3,2,2,5};
```

6.4 Examples Using Arrays

```
/* for each response, get rating and increment appropriate bin*/
for (int resp_cnt = 0; resp_cnt < RESPONSE_SIZE; resp_cnt++ ) {
    rating = responses[ resp_cnt ]; //get response from array
    //note:rating-1 since array starts at 0 and ratings at 1
    frequency[rating-1]++; //select bin, add one to the bin
} // end for
// display results
printf( "%s%12s\n", "Rating", "Occurrences" );
// output the frequencies in a tabular format
//note:rating_cnt + 1 since array starts at 0 and ratings at 1
for (int rating_cnt = 0; rating_cnt < FREQUENCY_SIZE ; rating_cnt++ )
    printf( "%6d%12d\n", rating_cnt + 1, frequency[ rating_cnt ] );
return 0; // indicates successful termination
} // end main
```

6.4 Examples Using Arrays

Output

Rating	Occurrences
1	3
2	5
3	7
4	2
5	3

Today

Arrays I

- Definition of arrays
- Declaration of arrays
- Examples

Next lecture

Arrays II

- Passing arrays to functions
- Searching arrays

Homework

- 1 Study Sections 6.1-6.4 in Deitel & Deitel
- 2 Do Self Review Exercises 6.1(a)-(c), 6.2(a)-(d), 6.3, 6.5(a)-(d)
- 3 Do Exercises 6.8(a)-(d), 6.12 in Deitel & Deitel